

Full Name: Krish Kumar Singh

E-Mail: krishkumarsinghpw@gmail.com

GitHub:Krish260653

LinkedIn:www.linkedin.com/in/krish-kumar-singh-24a161387

TITLE

Improving Efficiency and Robustness of Curve Intersection Computations in CGAL

The CGAL is a widely used open-source C++ library that provides efficient and reliable implementations of geometric algorithms and data structures. It is designed to support robust geometric computations required in domains such as computer graphics, computational geometry research, CAD systems, GIS, and robotics.

One of its key components, the 2D Arrangement package, enables the representation and manipulation of subdivisions of the plane induced by curves. It uses a flexible traits class design, where different curve families (such as linear segments, conic curves, and Bézier curves) are handled through specialized functors that implement geometric operations like intersection detection.

Problem:

Intersection detection is a fundamental operation in the 2D Arrangement package, but it poses significant challenges in terms of efficiency and robustness. The current implementation relies on traits classes that provide specialized logic for different curve families, leading to variations in performance and complexity. As the diversity of supported curves increases, intersection computations become computationally expensive due to: a. Redundant calculations during repeated intersection checks b. Lack of unified optimization strategies across curve types c. Handling of degenerate and edge cases (e.g., overlapping curves, tangential intersections) Additionally, robustness issues may arise due to numerical precision limitations and inconsistencies in handling special geometric configurations. These challenges can negatively impact the overall performance and reliability of applications built on top of CGAL.

Solution:

This project aims to improve both the efficiency and robustness of curve intersection computations in CGAL by optimizing the underlying geometric operations within the traits classes.

The proposed solution includes:

Algorithmic Optimization:

Introducing more efficient intersection strategies such as filtering techniques (e.g., bounding box checks) to eliminate unnecessary computations before performing exact calculations.

Refinement of Traits Classes:

Enhancing the design and implementation of traits class functors to reduce redundancy and improve consistency across different curve families.

Handling of Degenerate Cases:

Improving robustness by explicitly addressing edge cases such as overlapping segments, coincident curves, and near-degenerate configurations.

Support for Multiple Curve Families:

Developing optimized approaches tailored for linear segments, conic curves, and Bézier curves while maintaining a consistent interface.

Performance Benchmarking:

Establishing benchmarks to compare performance before and after optimization to ensure measurable improvements.

Impact

The proposed improvements to curve intersection computations in CGAL are expected to have a significant impact on both the performance and reliability of geometric processing workflows.

Improved Performance:

By reducing redundant computations and introducing optimized intersection strategies, the project will significantly decrease the time required for geometric operations in the 2D Arrangement package. This is especially beneficial for large-scale inputs involving complex curve arrangements.

Enhanced Robustness:

Strengthening the handling of degenerate and edge cases will improve numerical stability and ensure consistent behavior across different curve families, reducing the likelihood of computational errors.

Scalability Across Applications:

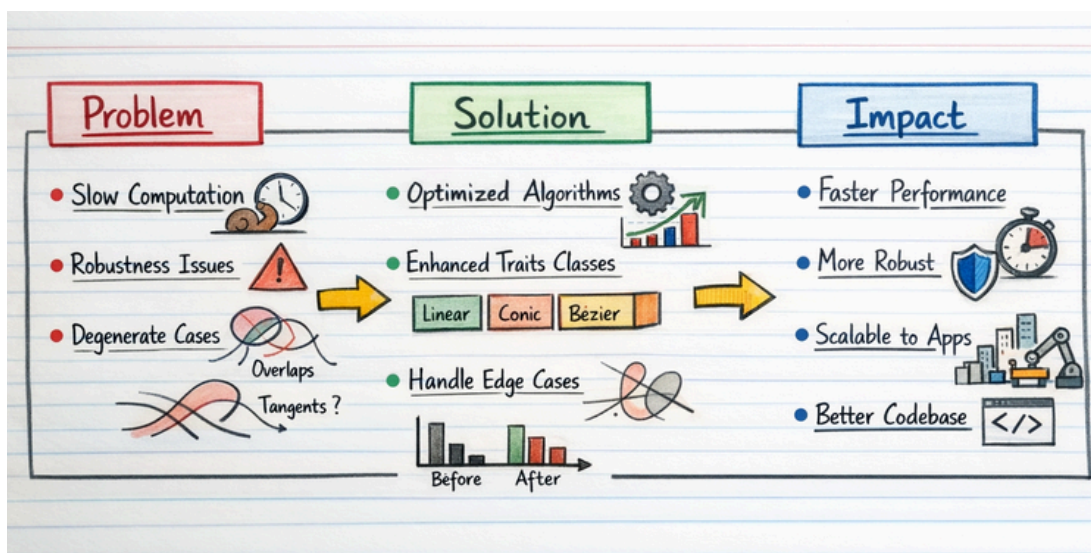
Faster and more reliable geometric operations will directly benefit applications in computer graphics, CAD systems, GIS, and robotics, where CGAL is widely used for precise geometric computations.

Better Developer Experience:

Cleaner and more consistent traits class implementations will make the codebase easier to understand, maintain, and extend for future contributors.

Long-Term Contribution to CGAL:

The optimizations introduced through this project will integrate into the core library, providing lasting value to the open-source community and enabling future enhancements in geometric algorithm design.



Introduction

The CGAL is a widely used C++ library for robust geometric computations, supporting applications in graphics, CAD, GIS, and robotics. A key component, the 2D Arrangement package, handles planar subdivisions induced by curves and enables operations such as insertion and intersection detection.

Key Concepts

2D Arrangement Package:

- a. Represents subdivisions of the plane using curves
- b. Core operations: insertion, traversal, intersection detection
- c. Backbone for many higher-level geometric algorithms

Traits Classes (Core Design):

--Define behavior of different curve types

Provide functors for operations like:

- a. Comparison
- b. Construction
- c. Intersection
- d. Enable flexibility without changing core structure

Supported Curve Families:

Linear Segments → simple, well-optimized

Conic Curves → more complex (algebraic)

Bézier Curves → parametric, numerically sensitive

Proposed Solution

This project aims to improve the efficiency and robustness of curve intersection computations in the 2D Arrangement package of CGAL by introducing algorithmic optimizations and enhancing the existing traits-based design.

1. Algorithmic Optimization of Intersection Computations:

To reduce computational overhead, the project will introduce efficient filtering and intersection strategies:

Bounding Box Filtering:

Perform fast preliminary checks using axis-aligned bounding boxes to eliminate non-intersecting curve pairs before invoking expensive exact computations.

Lazy Evaluation & Filtering Kernels:

Utilize CGAL's filtering mechanisms to first apply approximate computations and defer exact arithmetic only when necessary.

Efficient Pair Handling:

Reduce redundant intersection checks by improving how curve pairs are processed within the arrangement construction.

2. Enhancement of Traits Classes:

The traits class design will be refined to improve consistency and performance across curve families:

a. Optimize existing intersection functors to reduce repeated computations

b. Ensure uniform handling of geometric operations across different curve types

c. Improve modularity to allow easier extension and maintenance

This will strengthen the abstraction layer without modifying the core arrangement structure.

3. Curve-Specific Optimization Strategies:

a. Linear Segments:

Leverage existing kernel optimizations and remove redundant checks in intersection routines

b. Conic Curves:

Improve algebraic computation efficiency and handle precision-sensitive cases carefully

c. Bezier Curves:

Introduce subdivision or approximation-based techniques to simplify intersection detection while maintaining accuracy

4. Robust Handling of Degenerate Cases:

To improve reliability, the system will explicitly address challenging geometric scenarios:

- a. Overlapping and coincident curves
 - b. Tangential intersections
 - c. Near-degenerate configurations caused by floating-point inaccuracies
- Robust handling will ensure consistent and predictable behavior across all inputs.

5. Performance Benchmarking and Validation:

A comprehensive benchmarking framework will be developed to evaluate improvements:

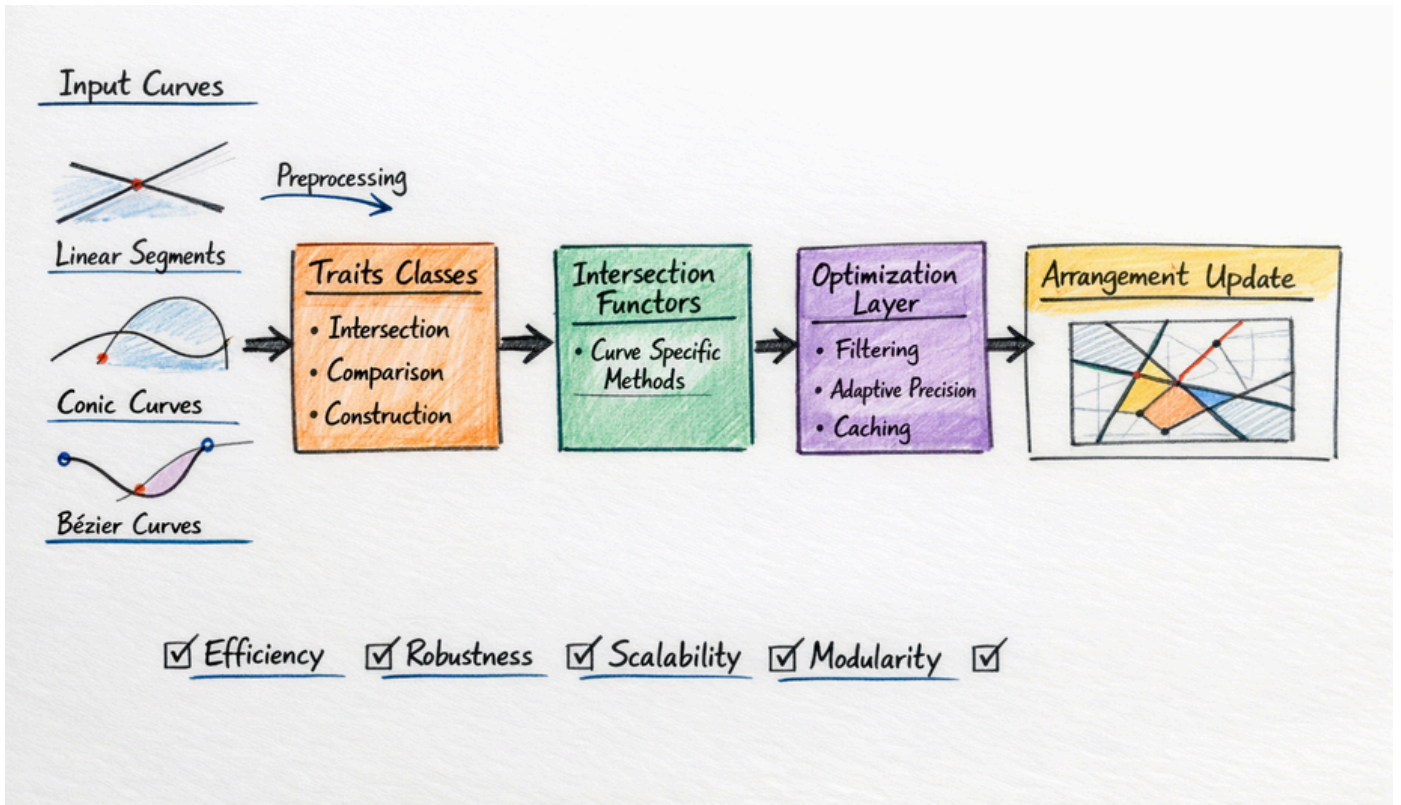
- a. Compare execution time before and after optimization
- b. Test across multiple curve families and input sizes
- c. Validate correctness using CGAL's existing test suite and additional edge-case scenarios

6. Integration with Existing CGAL Architecture:

All improvements will be designed to seamlessly integrate with CGAL's modular architecture:

- a. Maintain compatibility with existing APIs
- b. Follow CGAL coding standards and design principles
- c. Ensure that enhancements remain extensible for future curve types

System Architecture



Deliverables

- Optimized intersection algorithms for linear, conic, and Bézier curves within the CGAL*
- Enhanced and consistent traits class implementations for improved performance and maintainability*
- Robust handling of degenerate and edge cases (e.g., overlapping and tangential intersections)*
- Performance benchmarks demonstrating improvements over existing implementations*
- Comprehensive test cases ensuring correctness and stability*
- Integration of all improvements into the CGAL codebase with clean and maintainable code*
- Documentation explaining the implementation, optimizations, and usage*

Tech Stack

Programming Language: C++17

Library / Framework: CGAL

Core Concepts:

Computational Geometry, Geometric Algorithms,
Numerical Robustness

Development Tools:

Git, CMake

Testing & Benchmarking:

CGAL Test Suite, Custom Benchmarking Scripts

Risks and Mitigation

Complexity of Geometric Computations

Some intersection algorithms, especially for conic and Bézier curves, can be mathematically complex.

👉 *Mitigation: Study existing implementations in CGAL, refer to research literature, and regularly discuss approaches with mentors.*

Performance vs Robustness Trade-off

Optimizing for speed may affect numerical robustness in certain cases.

👉 *Mitigation: Use CGAL's exact and filtered kernels to balance performance and precision, supported by extensive testing.*

Handling Degenerate Cases

Edge cases such as overlapping or near-degenerate configurations may introduce unexpected behavior.

👉 *Mitigation: Design dedicated handling strategies and validate with comprehensive edge-case test scenarios.*

Integration Challenges

Ensuring compatibility with existing CGAL architecture and APIs may be challenging.

👉 *Mitigation: Follow CGAL coding standards, maintain modular changes, and perform incremental integration with continuous testing.*

Tech Stack :

Programming Language: C++17

Library / Framework: CGAL

Core Concepts:

Computational Geometry, Geometric Algorithms,
Numerical Robustness

Development Tools:

Git, CMake

Testing & Benchmarking:

CGAL Test Suite, Custom Benchmarking Scripts

Risks and Mitigation

Complexity of Geometric Computations

Some intersection algorithms, especially for conic and Bézier curves, can be mathematically complex.

👉 *Mitigation: Study existing implementations in CGAL, refer to research literature, and regularly discuss approaches with mentors.*

Performance vs Robustness Trade-off

Optimizing for speed may affect numerical robustness in certain cases.

👉 *Mitigation: Use CGAL's exact and filtered kernels to balance performance and precision, supported by extensive testing.*

Handling Degenerate Cases

Edge cases such as overlapping or near-degenerate configurations may introduce unexpected behavior.

👉 *Mitigation: Design dedicated handling strategies and validate with comprehensive edge-case test scenarios.*

Integration Challenges

Ensuring compatibility with existing CGAL architecture and APIs may be challenging.

👉 *Mitigation: Follow CGAL coding standards, maintain modular changes, and perform incremental integration with continuous testing.*

Implementation Timeline (350 Hours)



• Community Bonding Period

- Study CGAL architecture
- Review 2D Arrangement package
- Discuss with mentors

Phase 1: Analysis & Baseline Setup (Weeks 1-2)

- Analyze current operations
- Create test cases

—————→ Baseline Report & Tests

Phase 2: Linear Segment Optimization (Weeks 3-5)

- Optimize line intersections
- Bounding box checks

—————→ Optimized Segment Module

Phase 3: Conic Curve Optimization (Weeks 6-8)

- Enhance conic intersections
- Improve precision

—————→ Enhanced Conic Handling

Phase 4: Bézier Curve Optimization (Weeks 9-11)

- Efficient Bézier logic
- Traits integration

—————→ Optimized Bézier Module

Phase 5: Robustness & Edge Cases (Weeks 12-13)

- Handle overlaps & tangents
- Manage degenerate cases

—————→ Robust Intersection System

Phase 6: Integration & API Refinement (Weeks 14-15)

- Integrate all optimizations
- Refactor traits classes

—————→ Fully Integrated System

Phase 7: Final Testing & Docs (Weeks 16-17)

- Benchmark & Validate
- Write Documentation

—————→ Final Implementation

Weeks: 1-2 → 3-5 → 6-8 → 9-11 → 12-13 → 16-17

WHY ME?

I am a strong candidate for this project due to my solid foundation in C++ and data structures, along with a growing interest in computational geometry and algorithm optimization. I am comfortable working with modern C++ (C++17) and have experience with problem-solving that involves efficiency and correctness, which are critical for geometric computations.

I have a keen interest in understanding complex systems and improving their performance. This aligns well with the goals of CGAL, where both efficiency and robustness are essential. I am particularly motivated to work on low-level geometric operations, as they form the backbone of many real-world applications.

Additionally, I am familiar with version control tools like Git and follow a disciplined development approach that includes testing, debugging, and writing maintainable code. I am committed to actively communicating with mentors, incorporating feedback, and ensuring steady progress throughout the coding period.

Beyond the GSoC timeline, I am interested in continuing my contributions to CGAL and further exploring computational geometry, making this not just a short-term project but a long-term engagement.